

Bài 23: Dependency Injection trong Java

Dependency Injection (DI) là kỹ thuật cung cấp các dependency cần thiết cho một object từ bên ngoài thay vì để object tự tạo dependency đó.

DI giúp code giảm sự phụ thuộc, dễ thay đổi implementation, dễ kiểm thử và dễ mở rộng.

1. Dependency là gì?

Dependency là một object mà class cần sử dụng để thực hiện nhiệm vụ.

Ví dụ:

```
class EmailService {
    public void send(String message) {
        System.out.println("Gửi email: " + message);
    }
}

class UserService {
    private EmailService emailService = new EmailService();

    public void register(String username) {
        emailService.send("User mới: " + username);
    }
}
```

Trong ví dụ trên, UserService phụ thuộc vào EmailService.

2. Vấn đề khi tự tạo dependency

Khi class tự tạo dependency:

- Class bị liên kết chặt với một implementation cụ thể.
- Khó thay đổi EmailService thành SmsService.
- Khó viết unit test.
- Class vừa xử lý nghiệp vụ vừa chịu trách nhiệm tạo object.
- Khó quản lý vòng đời của dependency.

3. Dependency Injection là gì?

Thay vì tự tạo EmailService, UserService nhận EmailService từ bên ngoài:

```
class UserService {
    private final EmailService emailService;

    public UserService(EmailService emailService) {
```

```
        this.emailService = emailService;
    }

    public void register(String username) {
        emailService.send("User mới: " + username);
    }
}
```

Khởi tạo object:

```
EmailService emailService = new EmailService();
UserService userService = new UserService(emailService);
```

EmailService là dependency được inject vào UserService.

4. Inversion of Control

Inversion of Control (IoC) nghĩa là quyền kiểm soát việc tạo và cung cấp object được chuyển từ class sang một thành phần bên ngoài.

Không dùng IoC:

UserService tự tạo EmailService

Dùng IoC:

```
Application hoặc DI Container tạo EmailService
      ↓
    inject vào UserService
```

Dependency Injection là một cách phổ biến để thực hiện IoC.

5. Constructor Injection

Constructor Injection cung cấp dependency thông qua constructor.

```
interface MessageSender {
    void send(String message);
}

class UserService {
    private final MessageSender messageSender;

    public UserService(MessageSender messageSender) {
        this.messageSender = messageSender;
    }
}
```

```
    }

    public void register(String username) {
        messageSender.send("Đăng ký user: " + username);
    }
}
```

Implementation:

```
class EmailSender implements MessageSender {
    @Override
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}
```

Sử dụng:

```
MessageSender sender = new EmailSender();
UserService userService = new UserService(sender);

userService.register("An");
```

Ưu điểm:

- Dependency bắt buộc được cung cấp ngay khi tạo object.
- Field có thể khai báo final.
- Object luôn ở trạng thái hợp lệ.
- Dễ kiểm thử.
- Dependency được thể hiện rõ qua constructor.

6. Setter Injection

Setter Injection cung cấp dependency thông qua setter method.

```
class ReportService {
    private Printer printer;

    public void setPrinter(Printer printer) {
        this.printer = printer;
    }

    public void printReport(String content) {
        if (printer == null) {
            throw new IllegalStateException("Printer chưa được cấu hình");
        }
    }
}
```

```
        printer.print(content);  
    }  
}
```

Ưu điểm:

- Dependency có thể thay đổi sau khi object được tạo.
- Phù hợp với dependency không bắt buộc.

Nhược điểm:

- Có thể quên gọi setter.
- Object có thể ở trạng thái chưa hoàn chỉnh.
- Không thể khai báo dependency là final.

7. Field Injection

Field Injection gán dependency trực tiếp vào field, thường thông qua Reflection hoặc framework.

```
class OrderService {  
    @Inject  
    private PaymentService paymentService;  
}
```

Cách này thường thấy trong framework như Spring, nhưng không nên ưu tiên trong code thông thường.

Nhược điểm:

- Dependency bị ẩn.
- Khó tạo object bằng new trong unit test.
- Field không thể final.
- Object có thể chưa sẵn sàng sau constructor.
- Phụ thuộc nhiều vào framework.

8. So sánh các kiểu Injection

Kiểu	Cách cung cấp	Ưu điểm	Nhược điểm
Constructor Injection	Qua constructor	Rõ ràng, an toàn, dễ test	Constructor có thể nhiều tham số
Setter Injection	Qua setter	Dependency có thể thay đổi	Có thể quên cấu hình
Field Injection	Gán vào field	Viết ít code	Khó test, phụ thuộc framework

Thông thường nên ưu tiên Constructor Injection.

9. Dependency Inversion Principle

Dependency Inversion Principle (DIP) là nguyên tắc trong SOLID:

- Module cấp cao không nên phụ thuộc trực tiếp vào module cấp thấp.
- Cả hai nên phụ thuộc vào abstraction.
- Abstraction không nên phụ thuộc vào detail.
- Detail nên phụ thuộc vào abstraction.

Không tốt:

```
class OrderService {  
    private final MySQLOrderRepository repository =  
        new MySQLOrderRepository();  
}
```

Tốt hơn:

```
interface OrderRepository {  
    void save(String order);  
}  
  
class MySQLOrderRepository implements OrderRepository {  
    @Override  
    public void save(String order) {  
        System.out.println("Lưu order vào MySQL: " + order);  
    }  
}  
  
class OrderService {  
    private final OrderRepository repository;  
  
    public OrderService(OrderRepository repository) {  
        this.repository = repository;  
    }  
  
    public void createOrder(String order) {  
        repository.save(order);  
    }  
}
```

OrderService chỉ biết OrderRepository, không cần biết database cụ thể.

10. Thay đổi implementation

Nhờ phụ thuộc vào interface, có thể thay implementation mà không sửa OrderService.

```
class InMemoryOrderRepository implements OrderRepository {  
    @Override  
    public void save(String order) {  
        System.out.println("Lưu order vào bộ nhớ: " + order);  
    }  
}
```

```
OrderRepository repository = new InMemoryOrderRepository();  
OrderService service = new OrderService(repository);
```

Có thể thay MySQL bằng PostgreSQL, MongoDB, file hoặc fake repository.

11. DI và Unit Test

Không dùng DI:

```
class PriceService {  
    private final ProductRepository repository =  
        new MySqlProductRepository();  
}
```

Khi test, code luôn gọi MySQL thật.

Dùng DI:

```
interface ProductRepository {  
    double findPrice(String productCode);  
}  
  
class PriceService {  
    private final ProductRepository repository;  
  
    public PriceService(ProductRepository repository) {  
        this.repository = repository;  
    }  
  
    public double getPrice(String productCode) {  
        return repository.findPrice(productCode);  
    }  
}
```

Fake repository:

```
class FakeProductRepository implements ProductRepository {  
    @Override  
    public double findPrice(String productCode) {  
        return 100.0;  
    }  
}
```

Test:

```
ProductRepository fakeRepository = new FakeProductRepository();  
PriceService service = new PriceService(fakeRepository);  
  
double price = service.getPrice("JAVA");  
  
System.out.println(price); // 100.0
```

Không cần database thật để kiểm thử PriceService.

12. DI thủ công không cần framework

DI không bắt buộc phải dùng Spring.

Có thể tự tạo dependency trong class khởi động:

```
public class Application {  
    public static void main(String[] args) {  
        ProductRepository repository =  
            new MySqlProductRepository();  
  
        PriceService priceService =  
            new PriceService(repository);  
  
        priceService.getPrice("JAVA");  
    }  
}
```

Class Application đóng vai trò Composition Root, nơi các object được tạo và kết nối với nhau.

13. Composition Root

Composition Root là nơi tập trung việc:

- Tạo implementation.
- Tạo dependency.
- Kết nối các object.
- Cấu hình application.

```
public class Application {  
    public static void main(String[] args) {  
        MessageSender sender = new EmailSender();  
        UserService userService = new UserService(sender);  
  
        userService.register("An");  
    }  
}
```

Các class nghiệp vụ không cần tự tạo dependency.

14. DI trong Spring

Spring có IoC Container để tạo và inject object.

Interface:

```
public interface NotificationSender {  
    void send(String message);  
}
```

Implementation:

```
@Component  
public class EmailNotificationSender  
    implements NotificationSender {  
  
    @Override  
    public void send(String message) {  
        System.out.println("Email: " + message);  
    }  
}
```

Service:

```
@Service  
public class UserService {  
    private final NotificationSender sender;  
  
    public UserService(NotificationSender sender) {  
        this.sender = sender;  
    }  
  
    public void register(String username) {  
        sender.send("User mới: " + username);  
    }  
}
```



```
}  
}
```

Spring sẽ:

1. Tạo EmailNotificationSender.
2. Tạo UserService.
3. Tìm dependency NotificationSender.
4. Inject EmailNotificationSender vào UserService.

15. Một số annotation Spring

Annotation	Mục đích
@Component	Bean tổng quát
@Service	Bean chứa logic nghiệp vụ
@Repository	Bean truy cập dữ liệu
@Controller	Bean xử lý request web
@RestController	Controller trả về dữ liệu REST
@Configuration	Class chứa cấu hình bean

@Service và @Repository là các stereotype annotation chuyên biệt của @Component.

16. Constructor Injection trong Spring

Ví dụ:

```
@Service  
public class OrderService {  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

Với Spring hiện đại, nếu class chỉ có một constructor thì thường không cần @Autowired.

17. @Bean và @Configuration

Có thể tạo dependency bằng Java configuration:

```
@Configuration  
public class AppConfig {  
    @Bean
```

```
public NotificationSender notificationSender() {  
    return new EmailNotificationSender();  
}  
  
@Bean  
public UserService userService(  
    NotificationSender notificationSender) {  
    return new UserService(notificationSender);  
}  
}
```

Spring đọc cấu hình và quản lý các object được khai báo bằng @Bean.

18. Qualifier khi có nhiều implementation

Nếu có nhiều implementation của cùng một interface:

```
@Component("emailSender")  
class EmailSender implements MessageSender {  
    @Override  
    public void send(String message) {  
        System.out.println("Email: " + message);  
    }  
}
```

```
@Component("smsSender")  
class SmsSender implements MessageSender {  
    @Override  
    public void send(String message) {  
        System.out.println("SMS: " + message);  
    }  
}
```

Có thể chọn implementation bằng @Qualifier:

```
@Service  
class NotificationService {  
    private final MessageSender sender;  
  
    public NotificationService(  
        @Qualifier("smsSender") MessageSender sender) {  
        this.sender = sender;  
    }  
}
```

19. Primary implementation

Có thể đánh dấu một implementation là mặc định:

```
@Primary
@Component
class EmailSender implements MessageSender {
}
```

Khi có nhiều MessageSender nhưng không có @Qualifier, Spring sẽ ưu tiên implementation có @Primary.

20. Bean Scope

Scope xác định vòng đời và số lượng instance của bean.

Scope	Ý nghĩa
singleton	Một instance trong Spring Container
prototype	Tạo instance mới mỗi lần yêu cầu
request	Một instance cho mỗi HTTP request
session	Một instance cho mỗi HTTP session
application	Một instance cho toàn bộ web application

Mặc định bean trong Spring có scope singleton.

21. Vòng lặp dependency

Vòng lặp dependency xảy ra khi:

```
A cần B
B cần A
```

Ví dụ:

```
class ServiceA {
    private final ServiceB serviceB;

    ServiceA(ServiceB serviceB) {
        this.serviceB = serviceB;
    }
}

class ServiceB {
    private final ServiceA serviceA;

    ServiceB(ServiceA serviceA) {
        this.serviceA = serviceA;
    }
}
```

```
}  
}
```

Đây thường là dấu hiệu thiết kế chưa tốt.

Cách xử lý:

- Tách trách nhiệm của class.
- Tạo abstraction trung gian.
- Dùng event hoặc callback.
- Chỉ dùng lazy injection khi thực sự cần.

22. DI và Service Locator

Service Locator:

```
class UserService {  
    public void register() {  
        EmailService service =  
            ServiceLocator.get(EmailService.class);  
  
        service.send("Đăng ký user");  
    }  
}
```

UserService tự tìm dependency từ một nơi toàn cục. Dependency bị ẩn trong method.

Dependency Injection:

```
class UserService {  
    private final MessageSender sender;  
  
    UserService(MessageSender sender) {  
        this.sender = sender;  
    }  
  
    public void register() {  
        sender.send("Đăng ký user");  
    }  
}
```

Dependency được thể hiện rõ trong constructor và dễ thay thế khi test.

23. DI và Immutability

Constructor Injection cho phép field dependency là final:

```
class ReportService {  
    private final ReportRepository repository;  
  
    ReportService(ReportRepository repository) {  
        this.repository = repository;  
    }  
}
```

Sau khi object được tạo, repository không thể bị gán sang object khác.

24. Khi nào nên dùng DI?

Nên dùng DI khi:

- Class phụ thuộc vào service hoặc repository khác.
- Muốn thay đổi implementation.
- Cần unit test độc lập.
- Ứng dụng có nhiều module.
- Dependency có vòng đời cần quản lý.
- Muốn giảm coupling.
- Cần cấu hình theo môi trường.

Ví dụ:

```
Development → InMemoryRepository  
Testing      → FakeRepository  
Production   → MySqlRepository
```

25. Khi nào không cần DI?

Không cần tạo dependency thông qua container nếu object:

- Là class đơn giản, không có dependency.
- Là value object như Money, Point hoặc Address.
- Chỉ chứa hàm static thuần túy.
- Có vòng đời ngắn và không cần thay implementation.

```
public record Point(int x, int y) {  
}
```

26. Điểm chính

- Dependency là object mà một class cần sử dụng.
- DI cung cấp dependency từ bên ngoài class.
- IoC chuyển quyền kiểm soát tạo object ra bên ngoài.

- Constructor Injection thường là lựa chọn ưu tiên.
- Nên phụ thuộc vào interface thay vì implementation cụ thể.
- DI giúp giảm coupling và tăng khả năng kiểm thử.
- Spring IoC Container tự động tạo và inject bean.
- Composition Root là nơi kết nối các dependency.
- Dependency cycle thường cho thấy thiết kế cần được cải thiện.

27. Thực hành tốt

- Ưu tiên Constructor Injection.
- Khai báo dependency là final nếu có thể.
- Phụ thuộc vào abstraction.
- Nếu constructor quá nhiều tham số, xem lại trách nhiệm của class.
- Không tạo dependency bằng new bên trong class nghiệp vụ nếu muốn dễ thay thế.
- Không lạm dụng framework cho những object đơn giản.
- Tránh circular dependency.
- Tập trung wiring object tại Composition Root.

28. Các lỗi thường gặp

Lỗi	Nguyên nhân
NullPointerException	Dependency chưa được inject
UnsatisfiedDependencyException	Container không tìm thấy bean phù hợp
NoSuchBeanDefinitionException	Chưa đăng ký implementation hoặc component
NoUniqueBeanDefinitionException	Có nhiều implementation nhưng chưa chọn bean
Circular dependency	Các bean phụ thuộc vòng lặp lẫn nhau
Không test được class	Class tự tạo dependency bằng new
Dependency bị ẩn	Dùng Service Locator hoặc Field Injection quá nhiều
Constructor quá dài	Class có quá nhiều trách nhiệm hoặc dependency

29. Ghi nhớ

Dependency Injection:
 Class cần dependency
 ↓
 Nhận dependency từ bên ngoài
 ↓
 Sử dụng dependency qua abstraction

Constructor Injection → dependency bắt buộc
 Setter Injection → dependency tùy chọn hoặc có thể thay đổi

Field Injection → ngắn gọn nhưng khó test và khó kiểm soát

Ví dụ cơ bản:

```
interface PaymentGateway {  
    void pay(double amount);  
}  
  
class OrderService {  
    private final PaymentGateway paymentGateway;  
  
    OrderService(PaymentGateway paymentGateway) {  
        this.paymentGateway = paymentGateway;  
    }  
  
    void checkout(double amount) {  
        paymentGateway.pay(amount);  
    }  
}
```

OrderService không tự tạo PaymentGateway. Dependency được cung cấp từ bên ngoài, nên có thể dùng nhiều implementation khác nhau.